



Pontificia Universidad Javeriana
Facultad de Ingenieria
Fundamentos de Ingenieria de Software

Informe de Postmortem

Reporte del ciclo, retrospectiva Starfish y PIP

Milestone 2: Gestión Básica de Flota y Patrones de Diseño

Equipo SYNTIX TECH

Profesor	Luis Gabriel Moreno Sandoval
Monitor	Juan Pablo Arias Buitrago
Fecha	19 de mayo de 2026
Repositorio	https://github.com/puj-course/FIS_2610_3517_G4

Documento construido a partir del postmortem original del milestone, sin recortar informacion, adaptado al formato institucional de la plantilla suministrada.

Integrantes

Nombre	GitHub	Rol Scrum / tecnico
Sebastian Ramirez Maldonado	@Sarm-m	Scrum Master
Sebastian Vargas	@juanvargax	Product Owner / Sprint Planner
Samuel Freile	@samuelfl680	Configuration Manager
Solon Losada	@solonlosada2006	DevOps Engineer
Sebastian Rodriguez Ramirez	@juserora	QA Lead

Tabla 1: Integrantes y roles del equipo SYNTIX TECH.

Tabla de contenido

Integrantes	1
1. Introduccion	4
2. Postmortem – Milestone 2: Gestión Básica de Flota y Patrones de Diseño	5
2.1. 1. Resumen Ejecutivo	5
2.2. 2. Contexto de Ingeniería y Negocio	5
2.3. 3. Revisión de Datos del Proceso y Calidad	5
2.3.1. Desempeño real vs. planeado	6
2.3.2. Lecciones aprendidas	6
2.4. 4. Reporte de Roles	6
2.4.1. 4.1 Reporte del Scrum Master – Sebastián Ramírez Maldonado (@Sarm-m)	6
2.4.2. 4.2 Reporte del Product Owner / Sprint Planner – Sebastián Vargas (@juanvargax)	6
2.4.3. 4.3 Reporte del Configuration Manager – Samuel Freile (@samuelff680)	7
2.4.4. 4.4 Reporte del Quality Assurance Lead – Sebastián Rodríguez Ramírez (@Juserora)	7
2.4.5. 4.5 Reporte del DevOps Engineer – Solón Losada (@solonlosada2006) .	7
2.5. 5. Identificación de Causa Raíz	7
2.6. 6. Retrospectiva	8
2.7. 7. Retrospectiva Starfish	8
2.7.1. Seguir haciendo Seguir Haciendo	8
2.7.2. Comenzar a hacer Comenzar a Hacer	9
2.7.3. Dejar de hacer Dejar de Hacer	9
2.7.4. Mas de Más de	10
2.7.5. Menos de Menos de	10
2.8. 8. Plan de Mejora para el Milestone 3	10
2.9. 9. Respuestas completas a las preguntas guía de la PPT de Postmortem	11
2.9.1. 9.1 Revisión del producto, esfuerzo y proceso	11
2.9.2. 9.2 Evaluación objetiva de roles	12
2.9.3. 9.3 Reporte del ciclo por responsabilidades	14
2.9.4. 9.4 Estrategia de desarrollo y atributos de calidad	14

2.9.5. 9.5 Administración de configuración, riesgos y reutilización 15

2.9.6. 9.6 Reflexión del trabajo en grupo 15

2.9.7. 9.7 Verificación Starfish: respuestas por eje 15

2.9.8. 9.8 PIP - Plan de Mejora del Proceso 16

1. Introduccion

Este informe presenta el postmortem del Milestone 2 del proyecto DriveControl / AutoMinder Enterprise, desarrollado por el equipo SYNTIX TECH en el marco de la asignatura Fundamentos de Ingenieria de Software. El documento conserva de forma integral la informacion del postmortem original y la reorganiza bajo el formato institucional suministrado, manteniendo el contenido de resumen ejecutivo, contexto, revision de calidad, evaluacion de roles, causa raiz, retrospectiva Starfish, plan de mejora y respuestas completas a las preguntas guia de la presentacion de postmortem.

El proposito de este reporte es dejar una evidencia clara del ciclo: que producto se produjo, que esfuerzo se invirtio, que proceso se siguio, que problemas aparecieron, cuales fueron sus causas, que aprendizajes surgieron y que acciones de mejora deben incorporarse en el ciclo siguiente o en proyectos futuros. La estructura facilita la lectura academica y permite defender el trabajo con trazabilidad metodologica, tecnica y de calidad.

2. Postmortem – Milestone 2: Gestión Básica de Flota y Patrones de Diseño

Proyecto: Drive Control / Syntix Tech **Stack:** MERN (MongoDB, Express, React, Node.js)
Equipo:

Nombre	Usuario	Rol
Sebastián Ramírez Maldonado	@Sarm-m	Scrum Master
Samuel Freile	@samuelff680	Configuration Manager
Sebastián Rodríguez Ramírez	@Juserora	Quality Assurance Lead
Solón Losada	@solonlosada2006	DevOps Engineer
Sebastián Vargas	@juanvargax	Product Owner / Sprint Planner

2.1. 1. Resumen Ejecutivo

El Milestone 2 marcó la transición más significativa del proyecto en términos de arquitectura de software: el equipo dejó atrás un enfoque de código espagueti para adoptar una estructura limpia y desacoplada mediante la aplicación de patrones de diseño del catálogo GoF. Se implementaron el Factory Method para la creación dinámica de modales de gestión de flota y el patrón Facade para simplificar el subsistema de alertas documentales. Adicionalmente, se integró el módulo de Validación RUNT con un simulador de escenarios temporales controlados. Los principales retos técnicos de este hito estuvieron asociados a la interoperabilidad entre entornos de desarrollo (Windows vs. Linux) y a la ambigüedad en las convenciones de nombres de archivos.

2.2. 2. Contexto de Ingeniería y Negocio

Desde la perspectiva de negocio, el Milestone 2 habilitó la funcionalidad central del sistema: la gestión dinámica de la flota vehicular y la simulación de respuestas del RUNT colombiano para validar el estado documental de los vehículos. Esto permitió demostrar al stakeholder que la plataforma podía procesar escenarios reales de documentos vencidos, vigentes y próximos a vencer, aportando valor concreto más allá de la autenticación establecida en el M1.

Desde la perspectiva de ingeniería, la decisión de introducir patrones GoF en este hito fue estratégica: el crecimiento funcional previsto para los milestones 3 y 4 requería una arquitectura capaz de extenderse sin generar deuda técnica acumulativa. El desacoplamiento logrado mediante Factory y Facade fue la inversión de calidad que soportó el desarrollo posterior.

2.3. 3. Revisión de Datos del Proceso y Calidad

2.3.1. Desempeño real vs. planeado

- La comunicación sincrónica del equipo disminuyó respecto al M1, pero el rendimiento individual aumentó gracias a la división del trabajo en sub-issues técnicas atómicas en GitHub (un cambio puntual = una issue).
- Se registraron retrabajos considerables por incompatibilidades de rutas de archivos entre entornos Windows y Linux, especialmente en el uso del alias @/ para rutas absolutas en el frontend.
- Los componentes de Toasts fueron desarrollados antes de que el ToasterLayout base estuviera disponible, generando una dependencia invertida que obligó a rehacer parte del trabajo.
- El módulo RUNT fue entregado con el simulador de escenarios temporales funcional, cubriendo los tres estados documentales requeridos.

2.3.2. Lecciones aprendidas

- La estandarización de convenciones de nombres de archivos no es un detalle menor: conflictos como `useAlert.js` vs. `useAlerts.js` generaron confusión en las importaciones y bugs difíciles de rastrear.
 - Construir sobre infraestructura que aún no existe (Toasts sin ToasterLayout) es una forma de deuda técnica anticipada que siempre genera retrabajo.
 - La interoperabilidad entre entornos debe definirse como un requisito técnico del equipo, no como un problema individual de cada desarrollador.
-

2.4. 4. Reporte de Roles

2.4.1. 4.1 Reporte del Scrum Master – Sebastián Ramírez Maldonado (@Sarm-m)

En este milestone, el Scrum Master implementó la práctica de dividir el trabajo en sub-issues atómicas en GitHub, lo cual tuvo un impacto positivo directo en la trazabilidad y en la capacidad de los integrantes para trabajar de forma más autónoma. Sin embargo, la disminución de la comunicación sincrónica, aunque mejoró la productividad individual, redujo la visibilidad colectiva del estado de la integración. Los problemas de interoperabilidad entre entornos no fueron detectados a tiempo porque no había un espacio de revisión conjunta donde estos impedimentos pudieran emerger. Para el M3, el Scrum Master debe establecer al menos una sesión de sincronización semanal enfocada específicamente en la revisión de impedimentos técnicos y de integración.

2.4.2. 4.2 Reporte del Product Owner / Sprint Planner – Sebastián Vargas (@juan-vargax)

El Product Owner logró mantener el foco del sprint en las funcionalidades de mayor valor para el negocio: los módulos de flota y el simulador RUNT. Sin embargo, no se definió con suficiente

anticipación el alcance de los menús de navegación y las rutas asociadas, lo que generó que diferentes integrantes construyeran componentes con supuestos distintos sobre la estructura de navegación. Para el M3, el Sprint Planner debe incluir en el backlog refinado la definición explícita del mapa de rutas y la estructura de menús como prerequisite de cualquier historia de usuario que involucre navegación.

2.4.3. 4.3 Reporte del Configuration Manager – Samuel Freile (@samuelf680)

El Configuration Manager identificó durante este milestone el problema crítico de interoperabilidad entre entornos Windows y Linux. Las rutas de archivos absolutas y el uso del alias `@/` generaron comportamientos distintos en los entornos de cada desarrollador, causando que código que funcionaba localmente en un entorno fallara en otro. Adicionalmente, la ambigüedad en los nombres de archivos (`useAlert.js` vs. `useAlerts.js`) no fue capturada por ningún mecanismo automático, evidenciando la ausencia de linters estrictos. Para el M3, el Configuration Manager debe liderar la configuración de ESLint con reglas de naming conventions y la estandarización del alias `@/` en todos los entornos del equipo.

2.4.4. 4.4 Reporte del Quality Assurance Lead – Sebastián Rodríguez Ramírez (@Juserora)

El QA Lead identificó el patrón de fallo más recurrente del milestone: el intento de probar flujos completos (Toasts) antes de que la infraestructura base (ToasterLayout) estuviera disponible. Este antipatrón refleja la ausencia de una secuencia de construcción definida en el sprint. El QA Lead también detectó la duplicación de componentes en layouts como consecuencia de la falta de comunicación entre integrantes sobre qué ya estaba construido. Para el M3, se debe establecer una sesión de kick-off técnico al inicio del sprint donde se defina explícitamente el orden de construcción de los componentes y sus dependencias.

2.4.5. 4.5 Reporte del DevOps Engineer – Solón Losada (@solonlosada2006)

Durante este milestone, el DevOps Engineer no contó aún con un pipeline de CI/CD activo, lo que permitió que los problemas de interoperabilidad entre entornos llegaran tarde al conocimiento del equipo. La ausencia de un entorno homogeneizado (Docker) fue identificada como una deuda técnica de infraestructura que se pagó con retrabajo durante el sprint. Para el M3, el DevOps Engineer debe priorizar la configuración de un entorno Docker Compose básico que garantice que todos los integrantes ejecuten el proyecto en condiciones idénticas, eliminando la clase de errores "funciona en mi máquina".

2.5. 5. Identificación de Causa Raíz

La causa raíz principal del Milestone 2 fue la **falta de estandarización en las convenciones de nombres de archivos y en la definición de interoperabilidad entre entornos de desarrollo**. Estas dos condiciones operaron de forma combinada: la ambigüedad en los nombres (`useAlert` vs. `useAlerts`) generó bugs de importación silenciosos, mientras que la ausencia de un entorno homogeneizado hizo que los problemas de alias de rutas (`@/`) se manifestaran de

forma inconsistente según el sistema operativo de cada desarrollador.

Como causa secundaria, la **construcción de componentes visuales sobre infraestructura base inexistente** (Toasts antes que ToasterLayout) evidenció la ausencia de una secuencia de construcción acordada, generando una dependencia invertida que obligó a rehacer trabajo ya completado.

2.6. 6. Retrospectiva

El Milestone 2 fue el sprint de mayor densidad técnica del proyecto hasta ese momento. El equipo asumió la responsabilidad de refactorizar una base de código que, de haberse dejado crecer sin intervención, habría comprometido la mantenibilidad del sistema en los hitos siguientes. La decisión de aplicar patrones GoF no fue cosmética: representó una apuesta deliberada por la calidad arquitectónica sobre la velocidad de entrega a corto plazo, y sus beneficios se harían sentir en los milestones 3 y 4.

Sin embargo, el sprint también reveló una brecha importante en la cultura técnica del equipo: la interoperabilidad entre entornos no estaba en el radar de nadie como un riesgo real. El problema de Windows vs. Linux apareció cuando ya estaba bloqueando la integración, no antes. Esto es una señal de que el equipo aún no tenía el hábito de mapear riesgos técnicos de infraestructura al inicio del sprint.

La práctica de sub-issues atómicas en GitHub fue el hallazgo positivo más claro del milestone. Aunque redujo la comunicación sincrónica, mejoró notablemente la trazabilidad y la autonomía individual. El reto para el equipo es encontrar el equilibrio entre la autonomía que da la trazabilidad individual y la visibilidad colectiva que da la comunicación de integración.

El equipo salió del M2 con una arquitectura más sólida, pero también con la certeza de que la calidad no se construye solo con buenos patrones de diseño: también requiere convenciones de equipo, entornos homogéneos y una secuencia de construcción explícita.

2.7. 7. Retrospectiva Starfish

Reglas aplicadas: cada frase inicia con un verbo en infinitivo; ninguna frase se repite entre ejes.

2.7.1. Seguir haciendo Seguir Haciendo

#	Frase	Justificación
1	Aplicar patrones de diseño GoF para desacoplar componentes con responsabilidades diferenciadas.	El Factory Method y el Facade demostraron ser inversiones de calidad que simplificaron la extensión del sistema en este mismo sprint.
2	Dividir el trabajo en sub-issues técnicas atómicas vinculadas a commits específicos.	Mejoró la trazabilidad del desarrollo y permitió a los integrantes trabajar con mayor autonomía sin perder el hilo del progreso colectivo.

#	Frase	Justificación
3	Usar simuladores para cubrir escenarios de borde que aún no tienen infraestructura real disponible.	El simulador RUNT permitió validar los tres estados documentados sin depender de una conexión real a la entidad de tránsito.
4	Documentar los diagramas UML de los patrones en tiempo real, a medida que se implementan.	Facilita la comprensión del diseño por parte de todos los integrantes y sirve como referencia para extensiones futuras.
5	Separar las iteraciones de construcción de las iteraciones de refactorización dentro del sprint.	Evita mezclar la deuda técnica con el desarrollo de nuevas funcionalidades, manteniendo el foco del equipo en cada fase.

2.7.2. Comenzar a hacer Comenzar a Hacer

#	Frase	Justificación
1	Estandarizar las convenciones de nombres de archivos mediante reglas de ESLint configuradas en el repositorio.	Los conflictos entre <code>useAlert.js</code> y <code>useAlerts.js</code> generaron bugs de importación que consumieron tiempo de depuración innecesario.
2	Definir el mapa de rutas y la estructura de menús al inicio del sprint, antes de comenzar el desarrollo de componentes dependientes.	La ausencia de esta definición generó que distintos integrantes asumieran estructuras de navegación incompatibles.
3	Involucrar al QA Lead desde el inicio del sprint para mapear dependencias entre componentes y definir el orden de construcción.	El antipatrón de Toasts sin <code>ToasterLayout</code> se hubiera prevenido con una revisión temprana de dependencias.
4	Configurar linters estrictos con reglas de naming conventions y formato de importaciones en todos los entornos.	Un linter correctamente configurado convierte los errores de convención en fallos automáticos, no en bugs silenciosos.
5	Definir la interoperabilidad entre entornos de desarrollo como un requisito técnico del equipo desde el inicio de cada milestone.	La compatibilidad entre Windows y Linux no debe ser un problema individual: debe ser una decisión colectiva y documentada.
6	Recolectar evidencias técnicas del proceso en tiempo real durante el sprint, no solo al cierre.	Las evidencias recopiladas a posteriori pierden el contexto y la precisión necesarios para el análisis del postmortem.

2.7.3. Dejar de hacer Dejar de Hacer

#	Frase	Justificación
1	Construir componentes visuales que dependen de infraestructura base que aún no ha sido implementada.	Los Toasts desarrollados sin <code>ToasterLayout</code> generaron una dependencia invertida que obligó a rehacer trabajo completado.
2	Mezclar responsabilidades de presentación y lógica de negocio en el mismo componente sin aplicar separación de concerns.	Dificulta la aplicación de patrones de diseño y aumenta el costo de mantenimiento del código.
3	Ignorar los conflictos de alias de rutas (@/) entre entornos como si fueran problemas individuales.	Son problemas de equipo que requieren una solución de equipo: un entorno homogeneizado.
4	Duplicar archivos en layouts diferentes sin verificar si ya existe un componente equivalente en el repositorio.	La duplicación introduce deuda técnica y genera inconsistencias visuales difíciles de rastrear.

#	Frase	Justificación
5	Navegar directamente por URLs hard-codeadas en lugar de usar el sistema de enrutamiento definido.	Rompe la coherencia del sistema de navegación y genera dependencias ocultas que complican los cambios futuros.

2.7.4. Mas de Más de

#	Frase	Justificación
1	Realizar commits y merges frecuentes hacia la rama de desarrollo para visibilizar el progreso real del sprint.	Los commits frecuentes reducen el tamaño de los conflictos de integración y permiten detectar incompatibilidades más rápido.
2	Comunicar explícitamente al equipo cualquier cambio en los esquemas de datos o contratos de API.	Un cambio silencioso en el esquema puede romper componentes desarrollados por otros integrantes en paralelo.
3	Aplicar peer-feedback técnico en las revisiones de código, más allá de verificar que el código compila.	La revisión funcional y arquitectónica de los PRs es tan importante como la verificación de compilación.
4	Realizar una checklist de integración del Dashboard antes de cerrar cualquier historia de usuario que afecte la interfaz principal.	El Dashboard es el punto de convergencia de múltiples módulos; cualquier regresión allí es visible para el stakeholder.
5	Usar herramientas de diseño responsivo desde el inicio del desarrollo de componentes visuales.	Las correcciones de responsividad tardías son más costosas que diseñar con responsividad desde el primer commit.

2.7.5. Menos de Menos de

#	Frase	Justificación
1	Generar ambigüedad en los nombres de archivos mediante variaciones mínimas que inducen errores de importación.	Un carácter de diferencia en el nombre de un archivo puede generar horas de depuración innecesaria.
2	Posponer el feedback visual de los componentes hasta etapas avanzadas del sprint.	El feedback tardío sobre la interfaz genera retrabajo en código que ya pasó por revisión y fue marcado como completo.
3	Generar retrabajo por desalineación de entornos que podría haberse prevenido con una configuración compartida.	Docker Compose es la solución estructural a esta clase de problemas; su ausencia en este milestone fue costosa.
4	Depender de datos ingresados manualmente para probar flujos que deberían tener datos de prueba automatizados.	Los datos manuales son inconsistentes entre sesiones y generan falsos negativos en las pruebas.
5	Asumir que el código que funciona en un entorno funcionará en todos los entornos sin verificación explícita.	Esta suposición fue la fuente principal de los problemas de interoperabilidad Windows/Linux del milestone.

2.8. 8. Plan de Mejora para el Milestone 3

Acción	Responsable	Criterio de éxito
Configurar ESLint con naming conventions y reglas de importación en el repositorio.	Configuration Manager (@samuel680)	El pipeline rechaza automáticamente código que viola las convenciones.
Implementar Docker Compose para homogeneizar el entorno de desarrollo.	DevOps Engineer (@solonlosada2006)	Todos los integrantes ejecutan el proyecto con <code>docker compose up</code> sin configuración adicional.
Establecer sesión de kick-off técnico al inicio del sprint para mapear dependencias y orden de construcción.	Scrum Master (@Sarm-m)	Diagrama de dependencias de componentes generado antes del primer commit del M3.
Definir el mapa de rutas completo del sistema como prerequisite del sprint planning.	Product Owner (@juanvargax)	Todas las historias de usuario del M3 refieren rutas definidas en el mapa.
Implementar proceso formal de reporte de hallazgos de QA con pasos de reproducción.	QA Lead (@Juserora)	Todos los bugs del M3 registrados en GitHub Issues con template estandarizado.

2.9. 9. Respuestas completas a las preguntas guía de la PPT de Postmortem

Esta sección se agrega como matriz de verificación para dejar respondidas de forma explícita todas las preguntas guía de la presentación de Postmortem y Retrospectiva Starfish. Las respuestas se formulan con hechos del milestone, evitando evaluar personas y concentrándose en el desempeño del trabajo, el producto, el proceso y las oportunidades de mejora.

2.9.1. 9.1 Revisión del producto, esfuerzo y proceso

Pregunta guía de la PPT	Respuesta aplicada al Milestone 2: Gestión Básica de Flota y Patrones de Diseño
¿Qué producto se produjo?	Se produjo la gestión básica de flota, la incorporación de patrones GoF como Factory Method y Facade, el simulador de Validación RUNT y una arquitectura más desacoplada para soportar funcionalidades posteriores.
¿Qué esfuerzo se invirtió para hacerlo?	El esfuerzo se orientó a refactorizar, dividir trabajo en sub-issues, implementar componentes de flota y resolver problemas de compatibilidad entre entornos Windows/Linux.
¿Qué proceso se siguió para hacerlo?	El proceso mejoró frente al M1 por el uso de sub-issues atómicas y mayor trazabilidad, pero aún faltó una sincronización técnica suficiente para detectar dependencias e incompatibilidades antes de integrar.
¿Cómo fue el desempeño real comparado con lo planeado?	El desempeño fue mejor que en M1 en arquitectura y trazabilidad, pero tuvo desviaciones por problemas de alias de rutas, nombres de archivos y construcción de componentes sobre infraestructura aún no disponible.

Pregunta guía de la PPT	Respuesta aplicada al Milestone 2: Gestión Básica de Flota y Patrones de Diseño
¿Qué lecciones se aprendieron de esta experiencia?	El equipo aprendió que los patrones de diseño ayudan a mantener el sistema, pero que la arquitectura también depende de convenciones, entornos homogéneos y orden de construcción.
¿Debería usarse un criterio diferente, a nivel individual o de equipo, para el futuro?	Sí. Se debe usar un criterio de cierre que incluya revisión de dependencias, estandarización de nombres, validación en entornos distintos y confirmación de que la infraestructura base existe antes de construir encima.
¿Dónde hay oportunidades para mejorar y por qué?	La oportunidad principal está en estandarizar convenciones técnicas con ESLint, Docker Compose, mapa de rutas y revisión de contratos antes de implementar componentes dependientes.
¿Dónde hubo problemas que deben corregirse para el próximo ciclo?	Se deben corregir conflictos de naming, incompatibilidad del alias @/, baja sincronización de integración y desarrollo de Toasts antes del ToasterLayout.
¿Qué hizo bien el equipo y en qué falló?	Hizo bien: introducir patrones GoF, usar sub-issues atómicas y entregar el simulador RUNT. Falló en: subestimar riesgos de interoperabilidad, duplicar componentes y construir dependencias en orden incorrecto
¿Sirvieron las mejoras propuestas previamente?	Sí. La mejora de M1 sobre trazabilidad empezó a servir porque las sub-issues hicieron más claro el avance; no obstante, la mejora sobre validación técnica todavía fue incompleta.
¿Cómo ha sido el desempeño comparado con otros equipos?	No se cuenta con datos verificables de otros equipos, por lo que no se afirma una comparación externa. La comparación válida es interna: frente al M1 hubo avance en arquitectura, trazabilidad y autonomía, pero persistieron problemas de integración que exigían madurez de DevOps.
¿Se deben modificar los objetivos y las metas?	Sí. Las metas deben incluir no solo funcionalidades, sino también objetivos técnicos medibles: linters, rutas estandarizadas y entorno homogéneo.
¿Cómo se deben modificar los procesos?	Agregar kick-off técnico, mapa de rutas antes del desarrollo, checklist de dependencias y definición de convenciones de nombres.
¿Cuáles son las áreas de más alta prioridad para analizar?	Interoperabilidad de entornos, naming conventions, orden de construcción de componentes, revisión de contratos de API y rutas.

2.9.2. 9.2 Evaluación objetiva de roles

Rol	¿Qué funcionó?	¿Dónde estuvieron los problemas?	¿Dónde se puede mejorar?	Objetivo de mejoramiento para el Milestone 3
Scrum Master / Liderazgo	La división en sub-issues mejoró la trazabilidad y la autonomía.	La menor comunicación sincrónica redujo visibilidad sobre bloqueos técnicos.	Crear sincronización semanal de integración.	Detectar impedimentos de interoperabilidad antes de que bloqueen PRs.
Product Owner / Planeación	Priorizó flota y simulador RUNT como funcionalidades de alto valor.	No quedó definido a tiempo el mapa de rutas y menús.	Refinar navegación como prerequisite de historias visuales.	Documentar rutas y menús antes de iniciar desarrollo.
Configuration Manager / Soporte	Identificó la causa de conflictos entre entornos y archivos similares.	No había reglas automáticas para naming ni alias.	Configurar ESLint y convención de importaciones.	Eliminar errores por nombres y rutas inconsistentes.
QA Lead / Calidad	Detectó dependencia invertida entre Toasts y ToasterLayout.	El QA llegó tarde a mapear dependencias.	Participar desde el inicio en el orden de construcción.	Definir mapa de dependencias técnicas en el kick-off.
DevOps Engineer / Proceso de entrega	Identificó la necesidad de entorno homogéneo.	No existía Docker Compose ni CI suficientemente preventivo.	Normalizar entorno de ejecución.	Reducir errores de tipo "funciona en mi máquina".
Ingenieros / Desarrollo	Implementaron patrones y módulos complejos.	Algunos componentes se crearon duplicados o dependientes de infraestructura inexistente.	Revisar catálogo de componentes y dependencias antes de codificar.	Construir componentes reutilizables y alineados con la arquitectura.

2.9.3. 9.3 Reporte del ciclo por responsabilidades

Responsabilidad solicitada en la PPT	Respuesta del milestone
Resumen del ciclo	Ciclo de maduración arquitectónica: el producto ganó estructura técnica, pero el proceso evidenció falta de estandarización entre entornos y convenciones.
Liderazgo: motivación, compromisos, reuniones y apoyo requerido	La motivación aumentó por avances técnicos visibles; falló la visibilidad de integración y se requiere coordinación de riesgos técnicos.
Desarrollo: contenido producido frente a requisitos	El producto cumplió con flota, patrones y RUNT, pero algunos enfoques de integración generaron retrabajo.
Planeación: tareas, seguimiento y compromisos	La planeación fue más granular gracias a sub-issues, pero no contempló dependencias entre componentes.
Proceso: disciplina de trabajo, medición y seguimiento	El proceso mejoró en trazabilidad, aunque todavía no medía riesgos técnicos de entorno.
Calidad: estándares, inspecciones, defectos y PIP	La calidad arquitectónica mejoró por patrones, pero la calidad operativa se afectó por naming, rutas y orden de construcción.
Soporte: logística, configuración, cambios y riesgos	La configuración no fue suficiente para garantizar consistencia entre Windows y Linux.
Reporte de ingenieros: planeado vs. ejecutado y mejora personal	Los ingenieros ejecutaron tareas más técnicas que en M1, pero deben validar compatibilidad y dependencia antes de cerrar.

2.9.4. 9.4 Estrategia de desarrollo y atributos de calidad

Pregunta guía	Respuesta
¿La estrategia de desarrollo funcionó?	Funcionó en términos de arquitectura, porque refactorizar con patrones evitó mayor deuda técnica. Falló parcialmente en coordinación de dependencias.
¿Qué otros enfoques hubieran sido más adecuados?	Hubiera sido más adecuado diseñar primero un mapa de componentes, rutas y dependencias, y luego asignar implementación.
¿Cómo debería cambiarse la estrategia en el futuro?	Mantener patrones, pero preceder el desarrollo con convenciones, linters, entorno compartido y revisión de dependencias.
¿Cómo se tuvo en cuenta la usabilidad?	Se avanzó en flujos de gestión, aunque la navegación y menús necesitaban definición más temprana.
¿Cómo se tuvo en cuenta la mantenibilidad?	Mejoró con Factory y Facade, reduciendo acoplamiento y facilitando extensión posterior.
¿Cómo se tuvo en cuenta la compatibilidad?	Fue el punto débil por diferencias Windows/Linux y alias de rutas.

Pregunta guía	Respuesta
¿Cómo se tuvo en cuenta el desempeño?	No hubo foco fuerte en medición de desempeño; el objetivo fue estabilidad funcional.
¿Cómo se tuvo en cuenta la seguridad?	El simulador RUNT y módulos de flota se enfocaron en lógica de negocio; la seguridad de configuración aún debía reforzarse en ciclos siguientes.
¿Cómo mejorar estos tópicos en el futuro?	Incluir pruebas de navegación, linters, validación multi-entorno, criterios de responsividad y revisión de seguridad de configuración.

2.9.5. 9.5 Administración de configuración, riesgos y reutilización

Pregunta guía	Respuesta
¿Funcionó la administración de configuración?	Funcionó parcialmente: Git permitió seguimiento, pero la configuración del entorno no estaba homogeneizada.
¿Funcionó el control de cambios?	Las sub-issues mejoraron el control, pero los cambios de nombres y rutas no tuvieron revisión automática.
¿Cómo mejorarlos?	Crear reglas de naming, estandarizar alias, definir Docker Compose y checklist de cambios de contratos.
¿Fue efectivo el manejo y seguimiento de riesgos?	El seguimiento de riesgos fue insuficiente porque la interoperabilidad no se identificó como riesgo al inicio.
¿Fue buena la estrategia de reutilización?	Fue positiva en intención por los patrones, pero debe evitar duplicación de componentes y layouts.

2.9.6. 9.6 Reflexión del trabajo en grupo

Aspecto de reflexión	Respuesta
Comunicación del grupo	La comunicación asíncrona permitió autonomía, pero faltó sincronización técnica.
Calidad del trabajo de los integrantes	La calidad del diseño mejoró, aunque no toda la calidad de integración.
Planeación de tareas y seguimiento	Las sub-issues ayudaron, pero faltó mapear dependencias.
Mitigación de riesgos	El riesgo de entorno apareció tarde y se volvió retrabajo.
Relación con el cliente / stakeholder académico	El valor de negocio se fortaleció con flota y RUNT, mostrando utilidad más allá de autenticación.
Relación con las tecnologías	El equipo empezó a usar patrones y aprendió que el stack requiere convenciones comunes.

2.9.7. 9.7 Verificación Starfish: respuestas por eje

Eje de la estrella	Pregunta de la PPT que responde	Respuesta concreta del milestone
Comenzar a hacer	¿Qué acciones deberíamos comenzar para lograr el éxito, resolver problemas, mitigar riesgos, mejorar calidad, aprender, usar herramientas, distribuir tiempo y comunicar mejor?	Estandarizar ESLint, mapa de rutas, orden de construcción y compatibilidad de entornos.
Más de	¿Qué acciones que ya hacemos deberíamos hacer más para mejorar productividad, comunicación, calidad y aprendizaje?	Hacer más commits visibles, comunicación de contratos, peer-feedback técnico y revisión de integración.
Seguir haciendo	¿Qué acciones que ya hacemos bien deberíamos continuar?	Mantener patrones GoF, sub-issues, simuladores y documentación UML vinculada al código.
Menos de	¿Qué acciones deberíamos hacer menos porque causan inconvenientes o reducen calidad, comunicación o planeación?	Reducir ambigüedad de nombres, feedback visual tardío y dependencia de datos manuales.
Dejar de hacer	¿Qué acciones deberíamos dejar de hacer porque no son productivas o ya no se necesitan?	Dejar de construir componentes sin infraestructura base, duplicar layouts e ignorar conflictos de alias.

2.9.8. 9.8 PIP - Plan de Mejora del Proceso

Mejora priorizada	Problema que ataca	Cambio concreto en el proceso	Evidencia esperada
Estandarizar nombres e importaciones	Bugs por <code>useAlert</code> vs. <code>useAlerts</code> y alias	Configurar ESLint y convención única	Pipeline/linter detecta inconsistencias
Kick-off técnico de dependencias	Toasts antes de <code>ToasterLayout</code>	Revisar orden de construcción antes de codificar	Mapa de dependencias aprobado
Entorno homogéneo	Diferencias Windows/Linux	Configurar Docker Compose o guía única de entorno	Ejecución reproducible en todos los equipos
Mapa de rutas	Supuestos distintos de navegación	Documentar rutas y menús por sprint	PRs alineados con rutas definidas